

Ejercicio 1 – Composite y Strategy

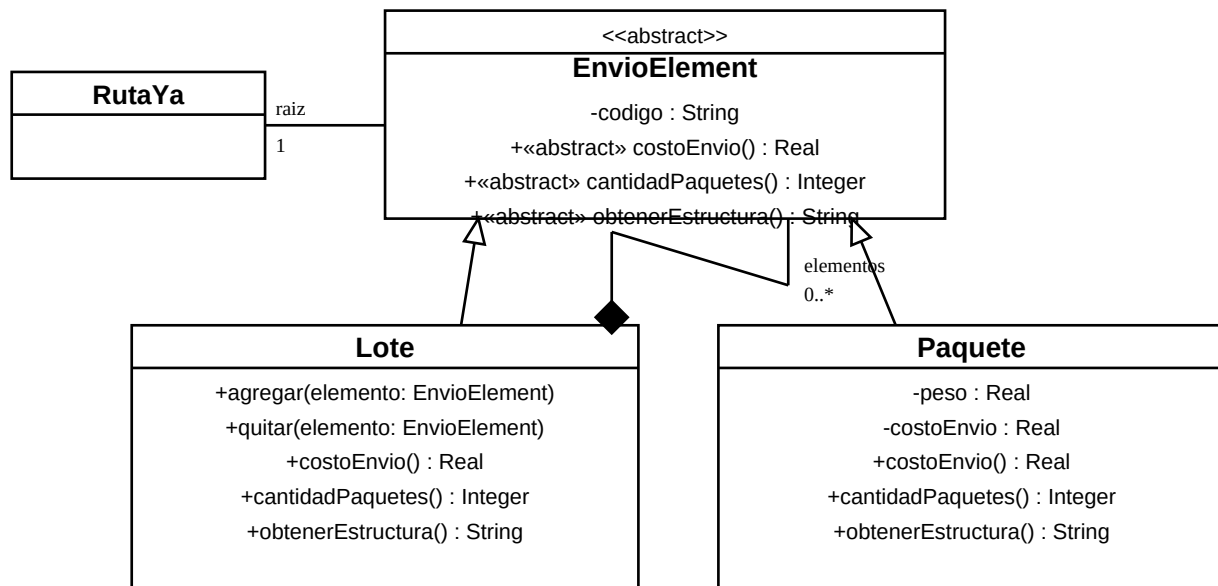
La empresa de logística **RutaYa** gestiona el cálculo de costos de envío de los paquetes que transporta entre distintas sucursales del país. Actualmente el sistema modela los envíos a partir de dos tipos de elementos:

- **Paquete:** unidad individual de envío, con un peso propio y un costo de envío ya calculado.
- **Lote:** agrupación de elementos de envío. Un lote puede contener tanto paquetes individuales como otros lotes (sublotes), permitiendo formar agrupaciones de cualquier profundidad.

La primera versión del sistema permite, para cualquier elemento de envío:

- Obtener el costo total de envío. El paquete retorna su propio costo, mientras que el lote retorna la suma de los costos de sus elementos.
- Conocer la cantidad total de paquetes involucrados.
- Consultar la estructura de lotes y sublotes.

El diagrama UML del sistema actual es el siguiente:



Actividades – parte 1:

1. Indique el nombre del patrón de diseño que se utiliza.
2. Explique con sus palabras (entre 2 y 5 renglones) cuál es el objetivo del patrón indicado en el inciso anterior.

RutaYa decidió avanzar con una nueva funcionalidad. Actualmente el costo de un paquete individual se calcula siempre de la misma forma, pero la empresa quiere poder elegir, para cada paquete, entre distintos

criterios de cálculo, como por ejemplo: (i) costo por peso, (ii) costo fijo según zona de destino, o (iii) costo expreso con recargo por urgencia. Pueden aparecer nuevos criterios en el futuro, y el criterio utilizado por un paquete puede cambiarse en cualquier momento, incluso luego de haber sido creado.

Actividades – parte 2:

3. Indique el nombre del patrón que aplicaría para incorporar la posibilidad de variar el criterio de cálculo del costo de un paquete.
4. Explique con sus palabras (entre 2 y 5 renglones) cuál es el objetivo del patrón indicado en el inciso anterior.
5. Realice el diagrama UML incorporando el patrón de diseño previamente definido, de modo que la clase Paquete calcule su costo delegando en el criterio elegido. Indique los roles en las clases utilizando estereotipos.

Espacio para el diagrama (punto 5)



Ejercicio 2 – Builder

La cadena de heladerías **FrioFrio** permite armar "copas heladas" personalizadas. Cada copa se compone de:

- Una base, a elección entre: helado, yogur o granizado (obligatoria, una sola).
- Entre 1 y 3 sabores (obligatorio al menos uno).
- Toppings opcionales, en cualquier cantidad y combinación (crocante, chocolate, frutas, chips de colores, entre otros).
- Crema y barquillo, ambos opcionales e independientes entre sí.

Para que una copa sea válida debe respetar estas reglas: siempre debe tener una base y al menos un sabor; el resto de los componentes es opcional y puede combinarse libremente. Actualmente, el código arma la copa mediante un único método con muchos parámetros, varios de ellos opcionales y frecuentemente nulos según el caso, lo que hace que el método sea difícil de leer, de mantener y propenso a errores al momento de invocarlo.

Además, FrioFrio quiere poder ofrecer "copas armadas" predefinidas con una composición fija (por ejemplo, "Copa Especial" o "Copa Tentación"), además de seguir permitiendo que un mozo arme una copa a medida, paso a paso, según lo que pida cada cliente.

Actividades:

1. Indique qué problema (code smell) presenta la forma actual de construir una copa. Justifique brevemente.
2. Indique qué patrón de diseño aplicaría para resolver el problema indicado y explique con sus palabras (entre 2 y 5 renglones) cuál es su objetivo.
3. Realice el diagrama de clases de la solución, indicando claramente los roles del patrón (por ejemplo: Director, Builder, ConcreteBuilder, Product) aplicados al caso de FrioFrio. Contemple en el diagrama tanto el armado de copas predefinidas como el armado a medida.

Espacio para el diagrama (punto 3)



Ejercicio 3 – Identificación de patrones

Considere cada uno de los siguientes casos y marque con una X el patrón que mejor aplica al problema descrito (no es necesario justificar).

Caso 1: Un sistema de pedidos modela un pedido que puede estar en distintos estados (Pendiente, En preparación, Enviado, Entregado, Cancelado). El comportamiento de operaciones como confirmar, cancelar o entregar cambia según el estado actual del pedido, y existen transiciones bien definidas entre esos estados.

Caso 2: Una aplicación de e-commerce necesita integrar las cotizaciones de un proveedor de envíos externo, cuya librería expone métodos y formatos de datos totalmente distintos a los que utiliza el sistema interno para calcular costos de envío. No es posible modificar la librería del proveedor.

Caso 3: Un sistema de facturación define un procedimiento general para emitir comprobantes (validar datos, calcular impuestos, generar el documento y enviarlo) que es el mismo para todos los tipos de comprobante, pero el cálculo de impuestos y el formato del documento cambian según el tipo (Factura, Nota de crédito, Recibo).

Caso 4: Una aplicación de mensajería permite agregar funcionalidades opcionales y combinables a un mensaje antes de enviarlo, como cifrado, compresión y firma digital, en cualquier combinación y orden, sin modificar la clase base de mensaje.

Caso 5: Un sistema necesita crear distintos tipos de notificación (Email, SMS, Push) según la configuración de cada usuario, y quiere centralizar y encapsular la lógica de creación para que el código cliente no quede lleno de condicionales eligiendo qué clase instanciar.

Caso	State	Adapter	Template Method	Decorator	Factory Method
Caso 1					
Caso 2					
Caso 3					
Caso 4					
Caso 5					

Ejercicio 4 – Verdadero o Falso

Responda verdadero (V) o falso (F) en cada caso:

- ☐ El patrón Adapter permite que dos interfaces incompatibles trabajen juntas sin modificar sus implementaciones existentes.
- ☐ Los patrones State y Strategy tienen una estructura de clases muy similar, pero se diferencian principalmente en su intención de uso.
- ☐ El patrón Composite permite tratar de manera uniforme a objetos individuales y a composiciones de objetos.
- ☐ El patrón Decorator es una alternativa a la herencia para extender funcionalidad de un objeto en tiempo de ejecución.
- ☐ El patrón Template Method permite que las subclases redefinan la estructura general del algoritmo, incluyendo el orden de sus pasos.
- ☐ El patrón Builder resulta especialmente útil cuando la construcción de un objeto tiene múltiples pasos opcionales o representaciones finales distintas.
- ☐ El patrón Factory Method delega en las subclases la decisión de qué clase concreta instanciar.

